

Lesson 12 DHT11 Temperature and Humidity Sensor

Overview

In this tutorial we will learn how to use a DHT11 Temperature and Humidity Sensor. It's accurate enough for most projects that need to keep track of humidity and temperature readings.

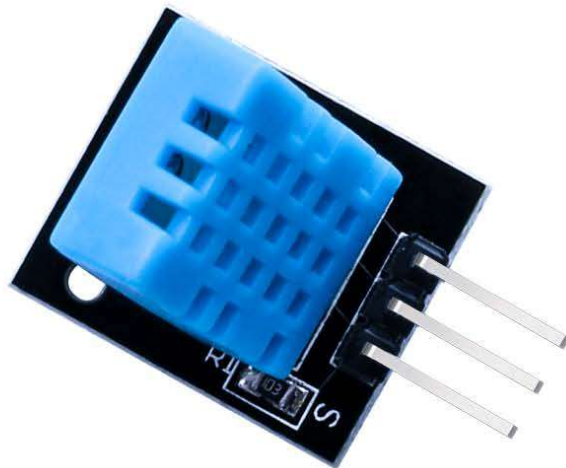
Again we will be using a Library specifically designed for these sensors that will make our code short and easy to write.

Component Required:

- (1) x Elegoo Mega 2560 R3
- (1) x DHT11 Temperature and Humidity module
- (3) x F-M wires (Female to Male DuPont wires)

Component Introduction

Temp and humidity sensor:



DHT11 digital temperature and humidity sensor is a composite Sensor which contains a calibrated digital signal output of the temperature and humidity. The dedicated digital modules collection technology and the temperature and humidity sensing technology are applied to ensure that the product has high reliability and

excellent long-term stability. The sensor includes a resistive sense of wet components and a NTC temperature measurement devices, and connects with a high-performance 8-bit microcontroller.

Applications: HVAC, dehumidifier, testing and inspection equipment, consumer goods, automotive, automatic control, data loggers, weather stations, home appliances, humidity regulator, medical and other humidity measurement and control.

Product parameters

Relative humidity:

Resolution: 16Bit

Repeatability: $\pm 1\%$ RH

Accuracy: At 25°C $\pm 5\%$ RH

Interchangeability: fully interchangeable

Response time: $1 / e$ (63%) of 25°C 6s

1m / s air 6s

Hysteresis: $< \pm 0.3\%$ RH

Long-term stability: $< \pm 0.5\%$ RH / yr in

Temperature:

Resolution: 16Bit

Repeatability: $\pm 0.2^{\circ}\text{C}$

Range: At 25°C $\pm 2^{\circ}\text{C}$

Response time: $1 / e$ (63%) 10S

Electrical Characteristics

Power supply: DC 3.5~5.5V

Supply Current: measurement 0.3mA standby 60 μA

Sampling period: more than 2 seconds

Pin Description:

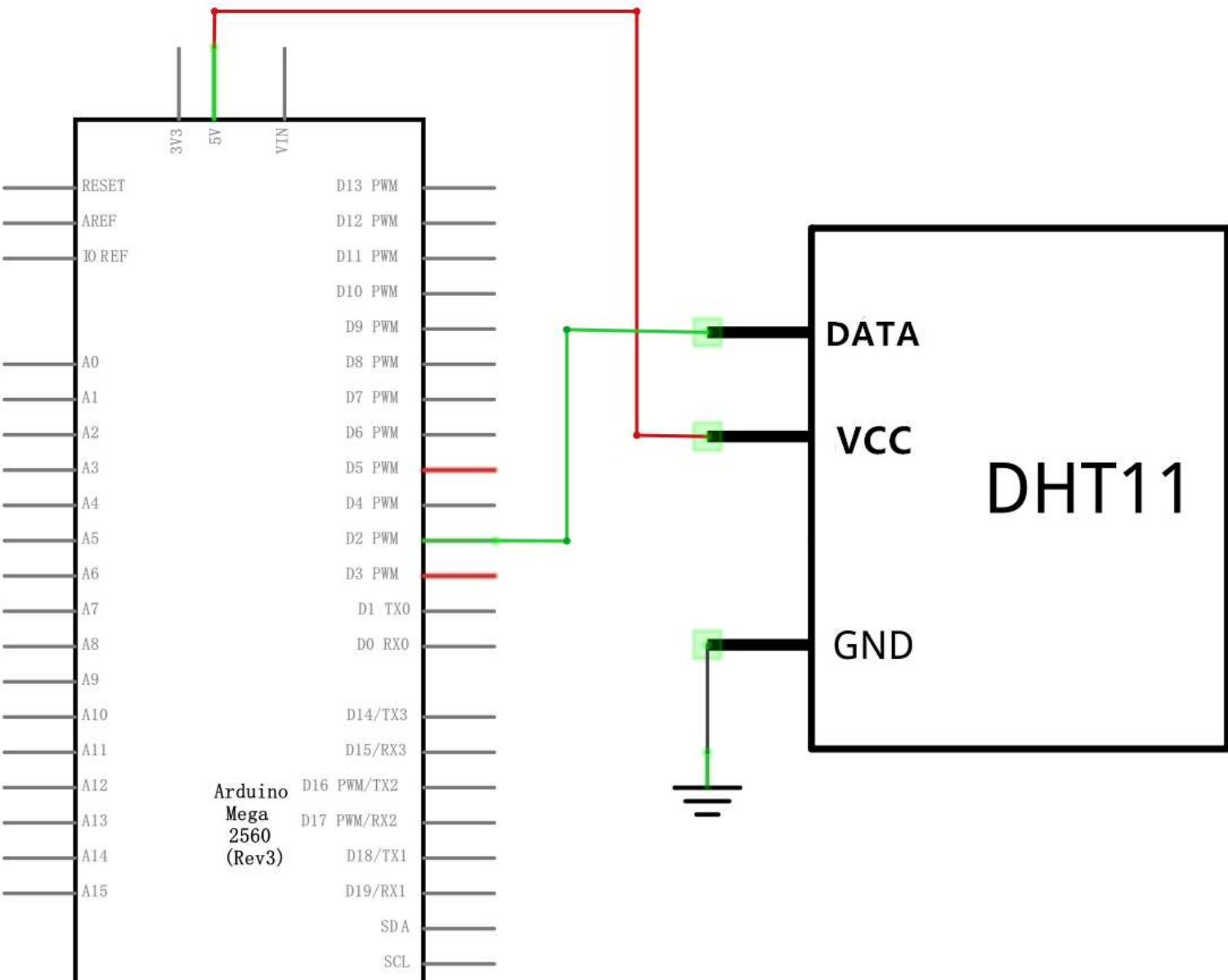
1, the VDD power supply 3.5~5.5V DC

2 DATA serial data, a single bus

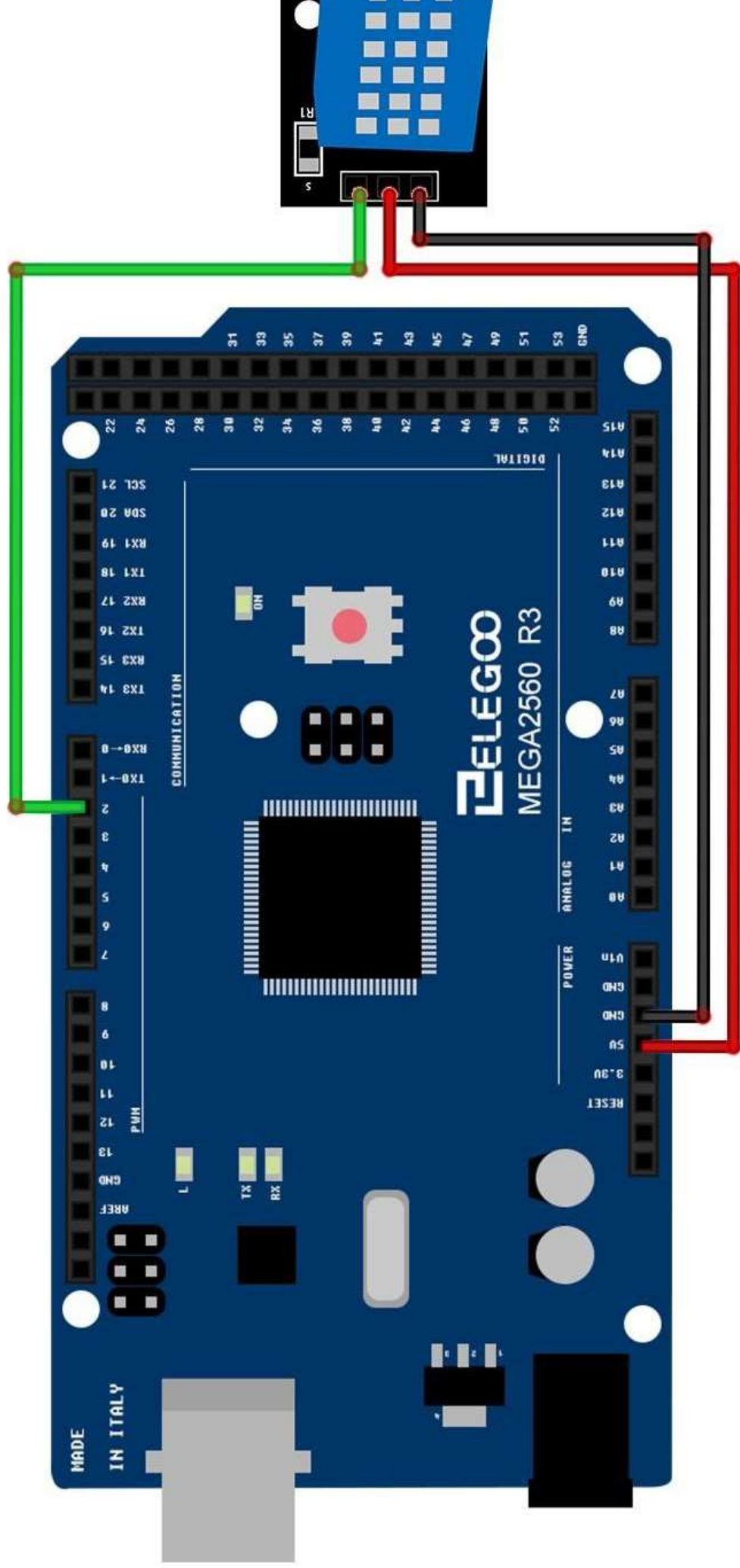
3, NC, empty pin

4, GND ground, the negative power

Connection
Schematic



Wiring diagram



Lesson 22 LCD Display

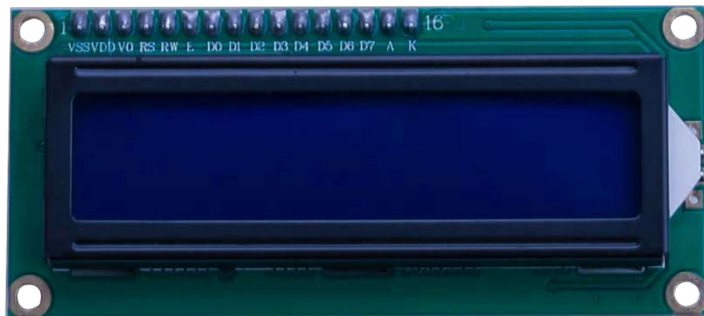
Overview

In this lesson, you will learn how to wire up and use an alphanumeric LCD display. The display has an LED backlight and can display two rows with up to 16 characters on each row. You can see the rectangles for each character on the display and the pixels that make up each character. The display is just white on blue and is intended for showing text.

In this lesson, we will run the Arduino example program for the LCD library, but in the next lesson, we will get our display to show the temperature, using sensors. |

Component Required:

- (1) x Elegoo Mega 2560 R3
- (1) x LCD1602 module
- (1) x Potentiometer (10k)
- (1) x 830 tie-points Breadboard
- (16) x M-M wires (Male to Male jumper wires)



Component Introduction

LCD1602

Introduction to the pins of LCD1602:

VSS: A pin that connects to ground

VDD: A pin that connects to a +5V powersupply

VO: A pin that adjust the contrast of LCD1602

RS: A register select pin that controls where in the LCD's memory you are writing data to. You can select either the data register, which holds what goes on the screen, or an instruction register, which is where the LCD's controller looks for instructions on what to do next.

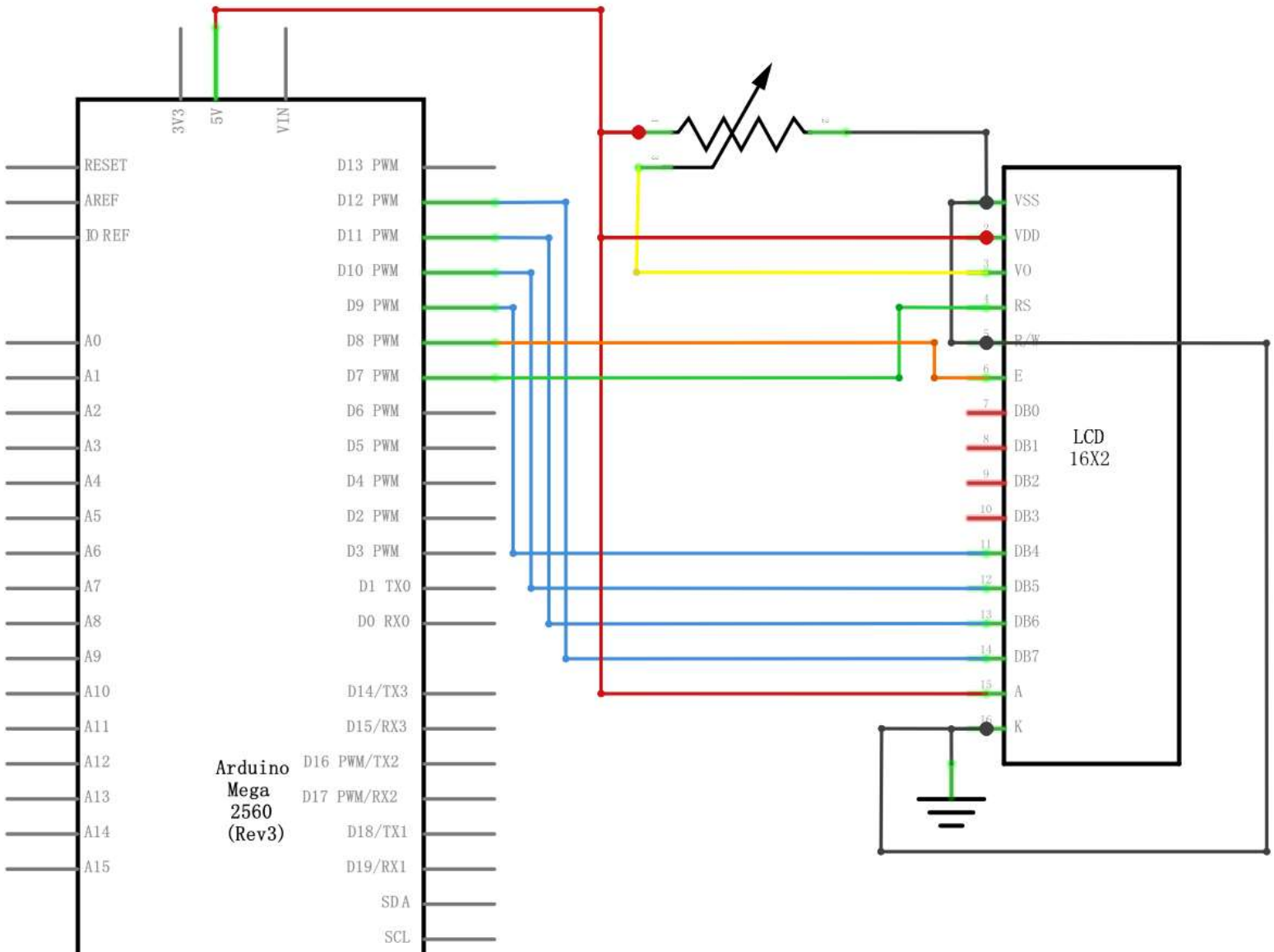
R/W: A Read/Write pin that selects reading mode or writing mode

E: An enabling pin that, when supplied with low-level energy, causes the LDC module to execute relevant instructions.

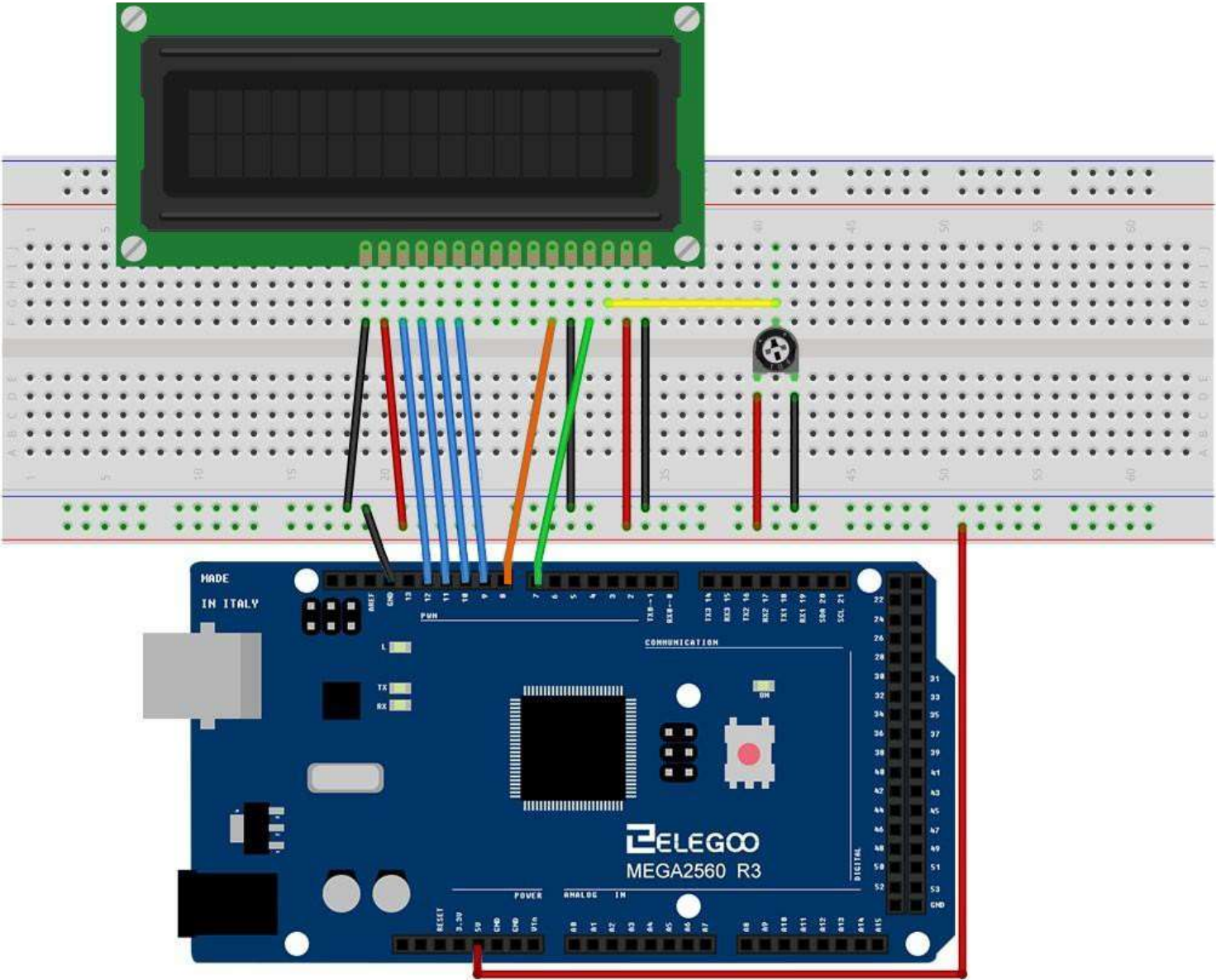
D0-D7: Pins that read and write data

A and K: Pins that control the LEDbacklight

Connection
Schematic



Wiring diagram



The LCD display needs six Arduino pins, all set to be digital outputs. It also needs 5V and GND connections.

There are a number of connections to be made. Lining up the display with the top of the breadboard helps to identify its pins without too much counting, especially if the breadboard has its rows numbered with row 1 as the top row of the board. Do not forget, the long yellow lead that links the slider of the pot to pin 3 of the display. The 'pot' is used to control the contrast of the display.

You may find that your display is supplied without header pins attached to it. If so, follow the instructions in the next section.

Code

After wiring, please open the program in the code folder- Lesson 22 LCD Display and click **UPLOAD** to upload the program. See Lesson 2 for details about program uploading if there are any errors.

Before you can run this, make sure that you have installed the `< Liquid Crystal >` library or re-install it, if necessary. Otherwise, your code won't work.

For details about loading the library file, see Lesson 1.

Upload the code to your Arduino board and you should see the message 'hello, world' displayed, followed by a number that counts up from zero.

The first thing of note in the sketch is the line:

```
#include <LiquidCrystal.h>
```

This tells Arduino that we wish to use the Liquid Crystal library.

Next we have the line that we had to modify. This defines which pins of the Arduino are to be connected to which pins of the display.

```
LiquidCrystal lcd(7, 8, 9, 10, 11, 12);
```

After uploading this code, make sure the backlight is lit up, and adjust the potentiometer all the way around until you see the text message

In the 'setup' function, we have two commands:

```
lcd.begin(16, 2);
```

```
lcd.print("Hello, World!");
```

The first tells the Liquid Crystal library how many columns and rows the display has.

The second line displays the message that we see on the first line of the screen.

In the 'loop' function, we also have two commands:

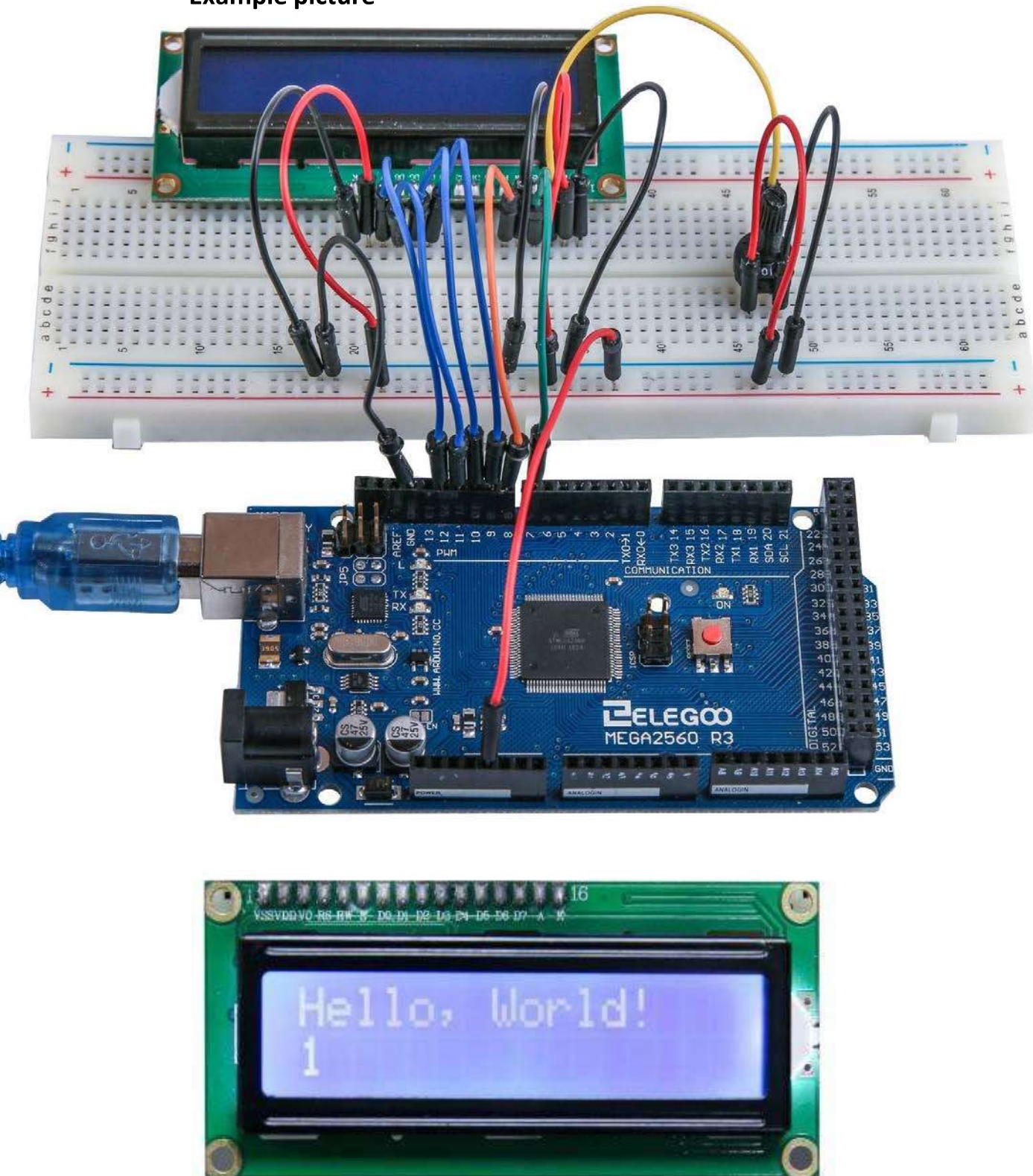
```
lcd.setCursor(0, 1);
```

```
lcd.print(millis()/1000);
```

The first sets the cursor position (where the next text will appear) to column 0 & row 1. Both column and row numbers start at 0 rather than 1.

The second line displays the number of milliseconds since the Arduino was reset.

Example picture



Lesson 23 Thermometer

Overview

In this lesson, you will use an LCD display to show the temperature.

Component Required:

- (1) x Elegoo Mega 2560 R3
- (1) x LCD1602 Module
- (1) x 10k ohm resistor
- (1) x Thermistor
- (1) x Potentiometer
- (1) x 830 tie-points Breadboard
- (18) x M-M wires (Male to Male jumper wires)

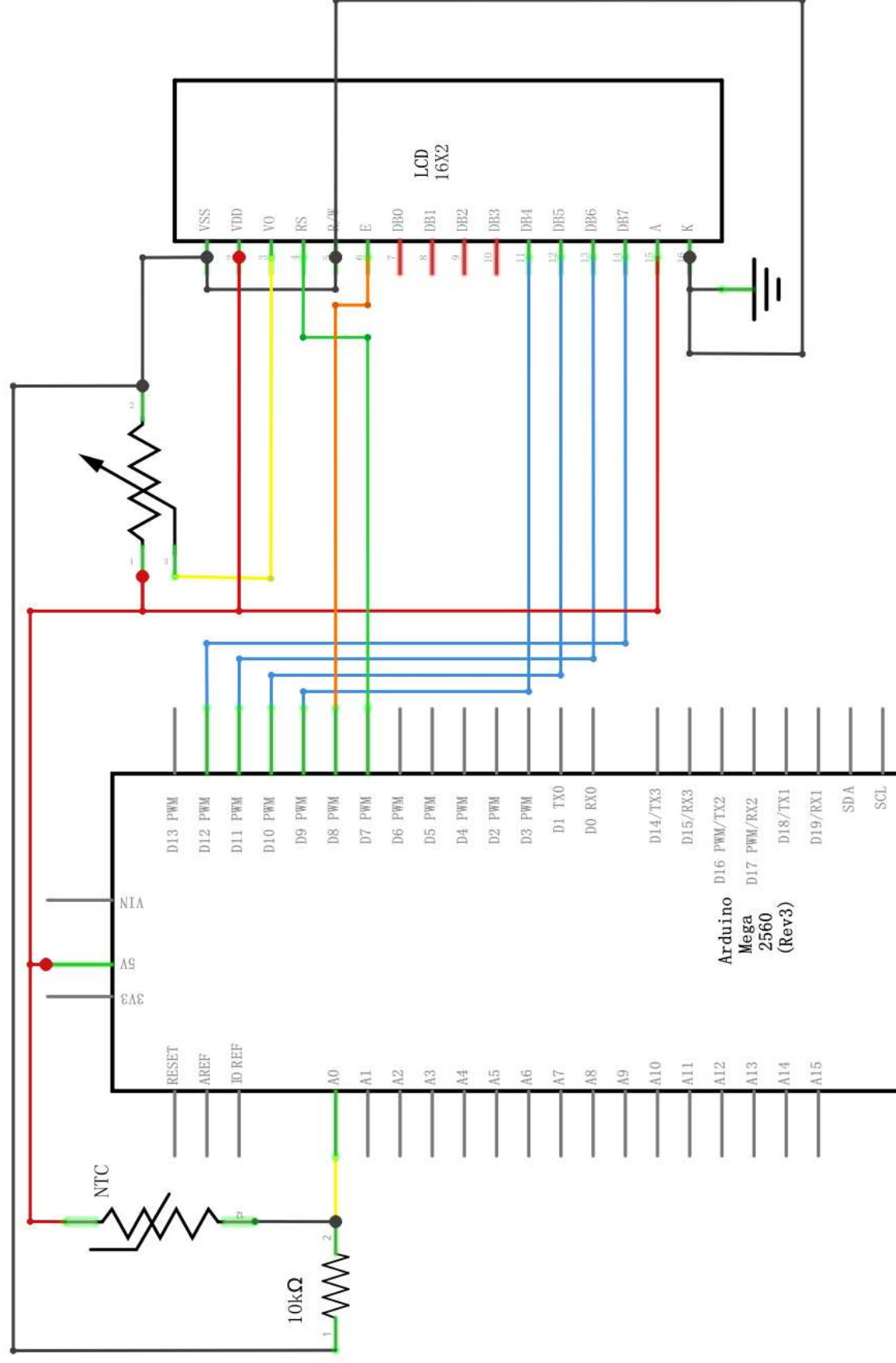
Component Introduction

Thermistor

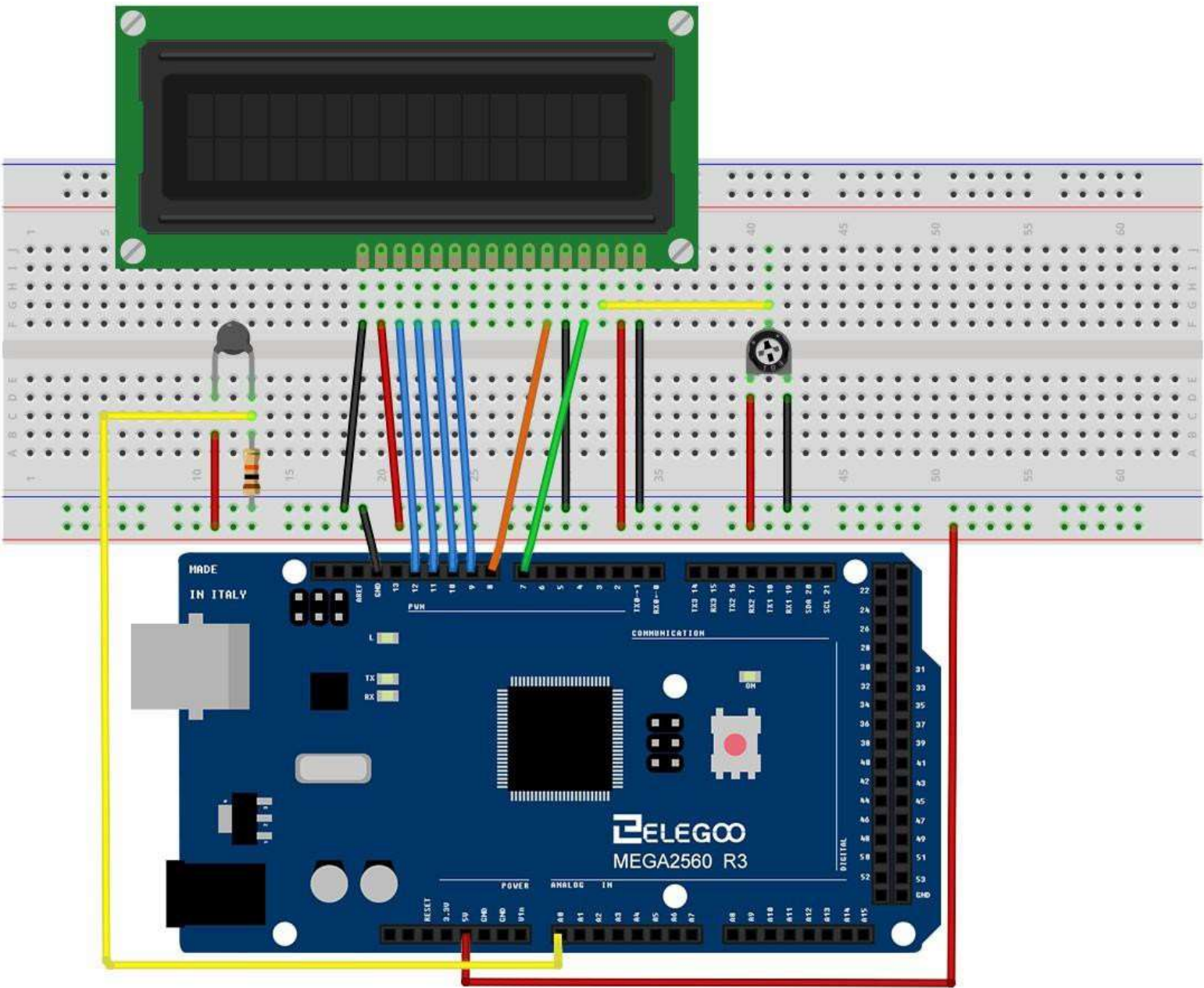
A thermistor is a thermal resistor - a resistor that changes its resistance with temperature. Technically, all resistors are thermistors - their resistance changes slightly with temperature - but the change is usually very small and difficult to measure. Thermistors are made so that the resistance changes drastically with temperature so that it can be 100 ohms or more of change per degree!

There are two kinds of thermistors, NTC (negative temperature coefficient) and PTC (positive temperature coefficient). In general, you will see NTC sensors used for temperature measurement. PTC's are often used as resettable fuses - an increase in temperature increases the resistance which means that as more current passes thru them, they heat up and 'choke back' the current, quite handy for protecting circuits!

Connection Schematic



Wiring diagram



The breadboard layout is based on the layout from Lesson 22, so it will simplify things if you still have this on the breadboard.

There are a few jumper wires near the pot that have been moved slightly on this layout.

The 10 kΩ resistor and thermistor are all new additions to the board.

Code

After wiring, please open the program in the code folder- Lesson 23 Thermometer and click UPLOAD to upload the program. See Lesson 2 for details about program uploading if there are any errors.

Before you can run this, make sure that you have installed the < Liquid Crystal > library or re-install it, if necessary. Otherwise, your code won't work.

For details about loading the library file, see Lesson 1.

The sketch for this is based on that of lesson 22. Load it up onto your Arduino and you should find that warming the temperature sensor by putting your finger on it will increase the temperature reading.

I find it useful to put a comment line above the 'lcd' command.

```
// BS E D4 D5 D6 D7  
LiquidCrystal lcd(7, 8, 9, 10, 11, 12);
```

This makes things easier if you decide to change which pins you use.

In the 'loop' function there are now two interesting things going on. Firstly we have to convert the analog from the temperature sensor into an actual temperature, and secondly we have to work out how to display them.

First of all, let's look at calculating the temperature.

```
int tempReading = analogRead(tempPin);  
double tempK = log(10000.0 * ((1024.0 / tempReading - 1)));  
tempK = 1 / (0.001129148 + (0.000234125 + (0.0000000876741 * tempK * tempK))  
* tempK );  
float tempC = tempK - 273.15;  
float tempF = (tempC * 9.0) / 5.0 + 32.0;
```

Displaying changing readings on an LCD display can be tricky. The main problem is that the reading may not always be the same number of digits. So, if the temperature changed from 101.50 to 99.00 then the extra digit from the old reading is in danger of being left on the display.

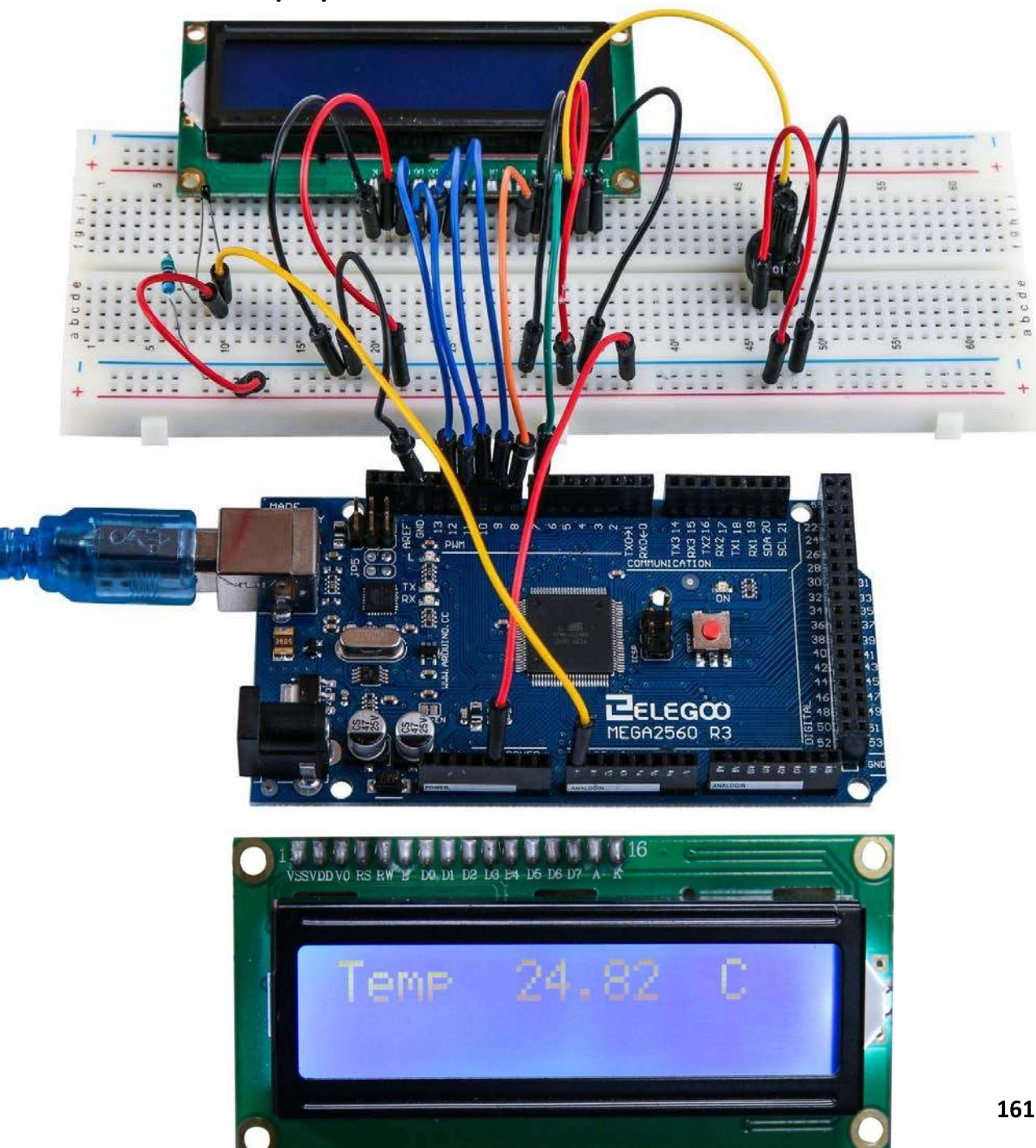
To avoid this, write the whole line of the LCD each time around the loop.

```
lcd.setCursor(0, 0);  
lcd.print("Temp      C  ");  
lcd.setCursor(6, 0);  
lcd.print(tempF);
```

The rather strange comment serves to remind you of the 16 columns of the display. You can then print a string of that length with spaces where the actual reading will go.

To fill in the blanks, set the cursor position for where the reading should appear and then print it.

Example picture



Lesson 31 Stepper Motor

Overview

In this lesson, you will learn a fun and easy way to drive a stepper motor.

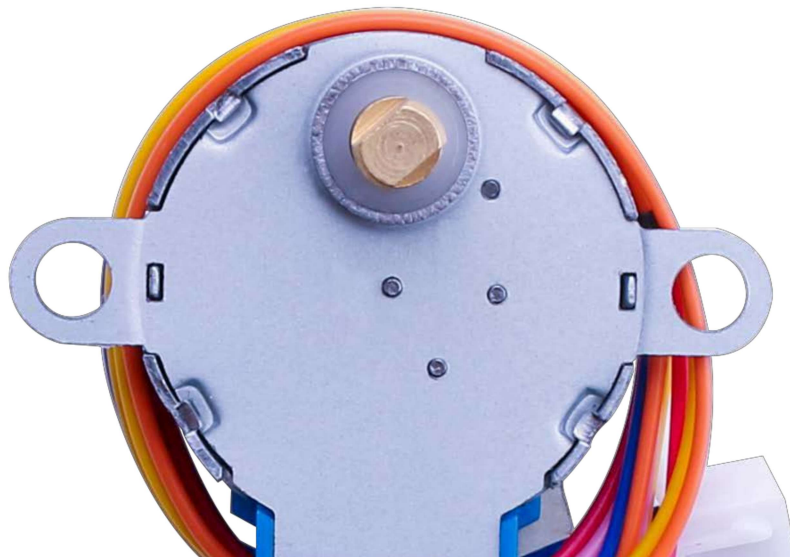
The stepper we are using comes with its own driver board making it easy to connect to our MEGA 2560.

Component Required:

- (1) x Elegoo Mega 2560R3
- (1) x 830 tie-points breadboard
- (1) x ULN2003 stepper motor driver module
- (1) x Stepper motor
- (1) x 9V1A Adapter
- (1) x Power supply module
- (6) x F-M wires (Female to Male DuPont wires)
- (1) x M-M wire (Male to Male jumper wire)

Component Introduction

Stepper Motor



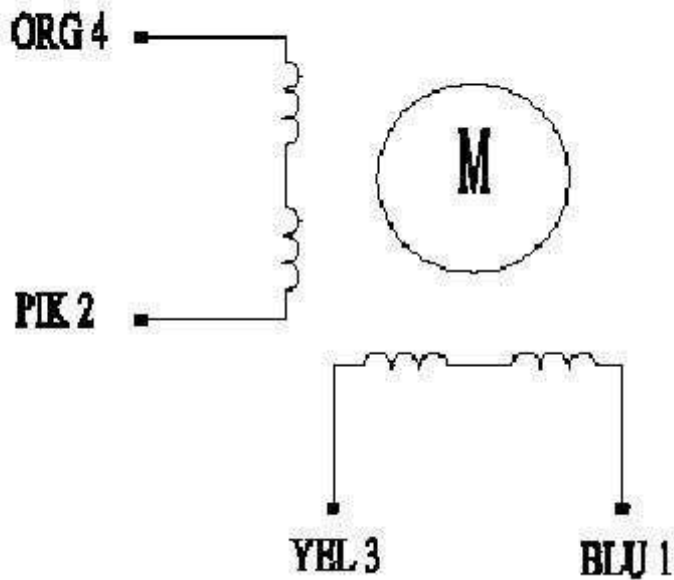
A stepper motor is an electromechanical device which converts electrical pulses into discrete mechanical movements. The shaft or spindle of a stepper motor rotates in discrete step increments when electrical command pulses are applied to it in the proper sequence. The motor's rotation has several direct relationships to these applied input pulses. The sequence of the applied pulses is directly related to the direction of motor shafts rotation. The speed of the motor shafts rotation is directly related to the frequency of the input pulses and the length of rotation is directly related to the number of input pulses applied. One of the most significant advantages of a stepper motor is its ability to be accurately controlled in an open loop system. Open loop control means no feedback information about position is needed. This type of control eliminates the need for expensive sensing and feedback devices such as optical encoders. Your position is known simply by keeping track of the input step pulses.

Stepper motor 28BYJ-48 Parameters

- Model: 28BYJ-48
- Rated voltage: 5VDC
- Number of Phase: 4
- Speed Variation Ratio: 1/64
- Stride Angle: $5.625^\circ / 64$
- Frequency: 100Hz
- DC resistance: $50\Omega \pm 7\% (25^\circ\text{C})$
- Idle In-traction Frequency: $> 600\text{Hz}$
- Idle Out-traction Frequency: $> 1000\text{Hz}$
- In-traction Torque $> 34.3\text{mN.m} (120\text{Hz})$
- Self-positioning Torque $> 34.3\text{mN.m}$
- Friction torque: 600-1200 gf.cm
- Pull in torque: 300 gf.cm
- Insulated resistance $> 10\text{M}\Omega (500\text{V})$
- Insulated electricity power: 600VAC/1mA/1s
- Insulation grade: A
- Rise in Temperature $< 40\text{K} (120\text{Hz})$
- Noise $< 35\text{dB} (120\text{Hz}, \text{No load}, 10\text{cm})$

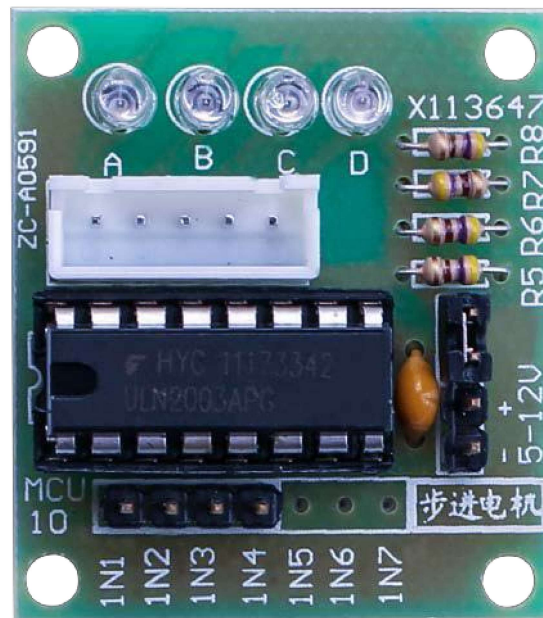
Interfacing circuits

WIRING DIAGRAM



The bipolar stepper motor usually has four wires coming out of it. Unlike unipolar steppers, bipolar steppers have no common center connection. They have two independent sets of coils instead. You can distinguish them from unipolar steppers by measuring the resistance between the wires. You should find two pairs of wires with equal resistance. If you've got the leads of your meter connected to two wires that are not connected (i.e. not attached to the same coil), you should see infinite resistance (or no continuity).

ULN2003 Driver Board



Product Description

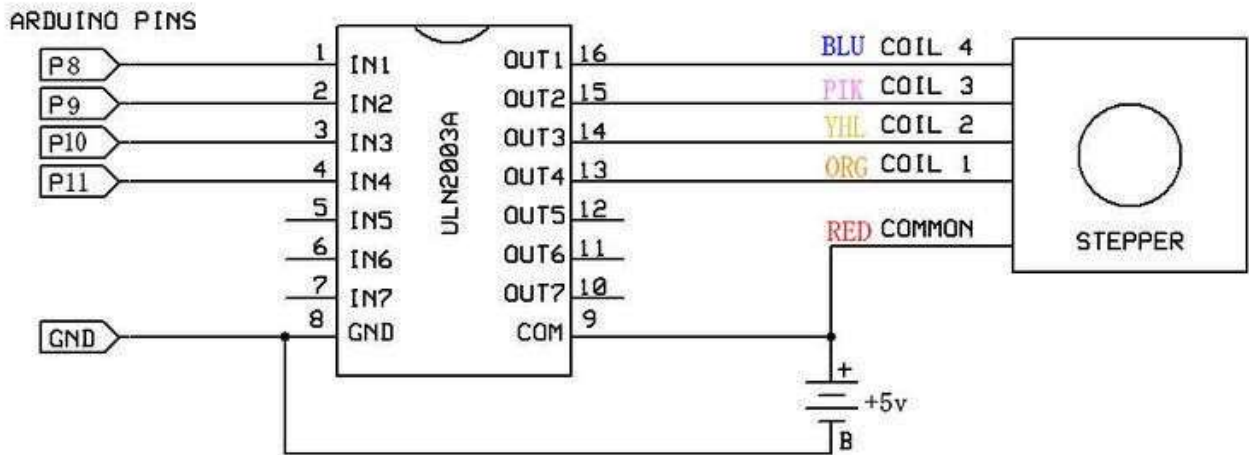
- Size: 42 mm x 30 mm
- Use ULN2003 driver chip, 500 mA
- A. B. C. D LED indicating the four phase stepper motor working condition.
- White jack is the four phase stepper motor standard jack.
- Power pins are separated
- We kept the rest pins of the ULN2003 chip for your further prototyping.

The simplest way of interfacing a unipolar stepper to Arduino is to use a breakout for ULN2003A transistor array chip. The ULN2003A contains seven darlington transistor drivers and is somewhat like having seven TIP120 transistors all in one package. The ULN2003A can pass up to 500 mA per channel and has an internal voltage drop of about 1V when on. It also contains internal clamp diodes to dissipate voltage spikes when driving inductive loads. To control the stepper, apply voltage to each of the coils in a specific sequence.

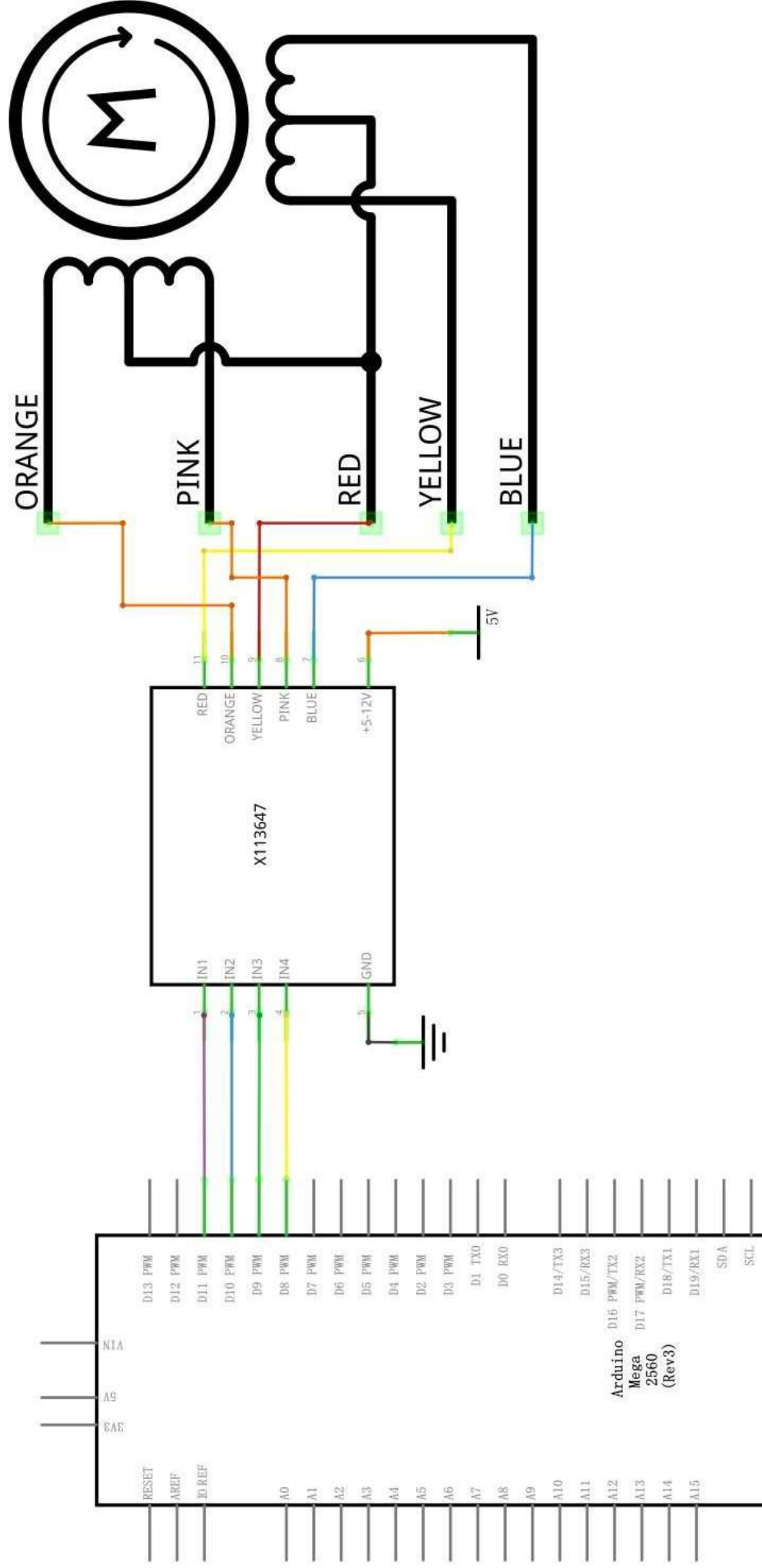
The sequence would go like this:

Lead Wire Color	---> CW Direction (1-2 Phase)								
	1	2	3	4	5	6	7	8	
4 ORG	-	-							-
3 YEL		-	-	-					
2 PIK				-	-	-			
1 BLU						-	-	-	

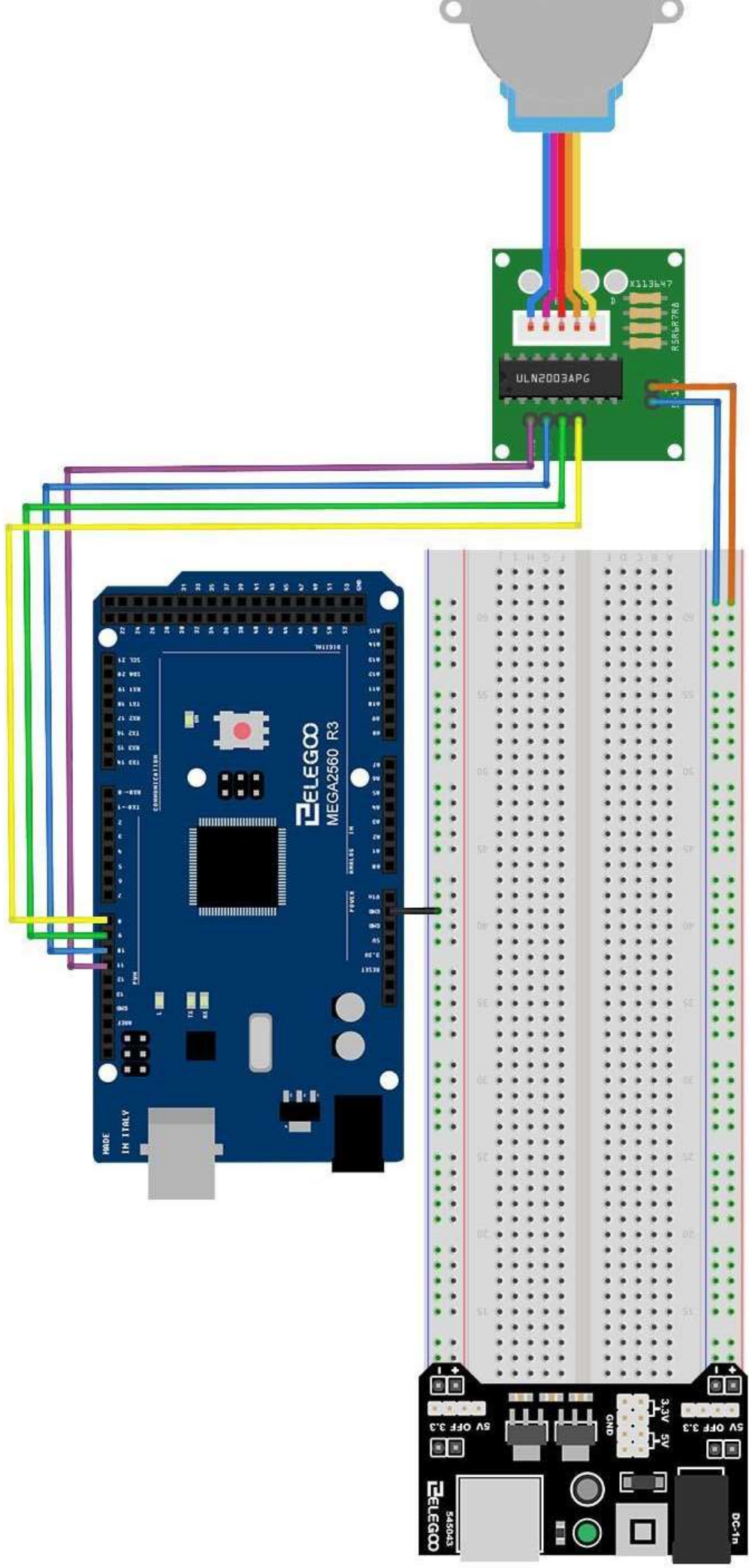
Here are schematics showing how to interface a unipolar stepper motor to four controller pins using a ULN2003A, and showing how to interface using four com



Connection Schematic



Wiring diagram



We are using 4 pins to control the Stepper.

Pin 8-11 are controlling the Stepermotor.

We connect the Ground from to MEGA 2560 to the Stepper motor.

Code

After wiring, please open the program in the code folder- Lesson 31 Stepper Motor and click UPLOAD to upload the program. See Lesson 2 for details about program uploading if there are any errors.

Before you can run this, make sure that you have installed the < Stepper > library or re-install it, if necessary. Otherwise, your code won't work.

For details about loading the library file, see Lesson 1.

Example picture

